

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce
Department of Information Technology (Bsc.IT Semester III)
SCALA for DS
Practical

<i>Roll No. : SY50</i>	<i>Name: Nitesh Yadav</i>
<i>Class:SYIT</i>	<i>Date of Submission :22 /09/2026</i>

27) Implement one-hot encoding for a categorical column in a dataset.

Input:

```
object Scala extends App {  
  val data = List("Red", "Blue", "Green", "Red", "Blue")  
  for (value <- data)  
  {  
    val r = if (value == "Red") 1 else 0  
    val b = if (value == "Blue") 1 else 0  
    val g = if (value == "Green") 1 else 0  
    println(value + " -> Red:" + r + " Blue:" + b + " Green:" + g)  
  }  
}
```

Output:

```
"C:\Program Files\Java\jdk1.8.0_181\bin  
Red -> Red:1 Blue:0 Green:0  
Blue -> Red:0 Blue:1 Green:0  
Green -> Red:0 Blue:0 Green:1  
Red -> Red:1 Blue:0 Green:0  
Blue -> Red:0 Blue:1 Green:0  
  
Process finished with exit code 0
```

28) Compute the Euclidean distance between two Breeze vectors. Use it for nearest neighbor classification.

Input:

```
build.sbt  Scala.scala x
1  import breeze.linalg._
2
3  ▶ object Scala extends App
4  {
5      val A = DenseVector(2.0, 3.0)
6      val B = DenseVector(5.0, 7.0)
7
8      val distance = norm(A - B)
9
10     println("Euclidean Distance: " + distance)
11
12     val C = DenseVector(2.5, 3.5)
13
14     val d1 = norm(C - A)
15     val d2 = norm(C - B)
16
17     if (d1 < d2)
18     |     println("Nearest Neighbor: A")
19     else
20     |     println("Nearest Neighbor: B")
21 }
```

Output:

```
Euclidean Distance: 5.0
Nearest Neighbor: A

Process finished with exit code 0
```

29) Create a simple Spark MLlib pipeline to classify data. Use logistic regression or decision trees.

Input:

```
import org.apache.spark.sql.SparkSession
import org.apache.spark.ml.feature.VectorAssembler
import org.apache.spark.ml.classification.LogisticRegression

object scala1 extends App {

  val spark = SparkSession.builder()
    .appName("Classification")
    .master("local[*]")
    .getOrCreate()

  import spark.implicits._

  val data = Seq(
    (1.0, 0.0),
    (2.0, 0.0),
    (3.0, 0.0),
    (7.0, 1.0),
    (8.0, 1.0),
    (9.0, 1.0)
  ).toDF("marks", "label")
```

```
val a = new VectorAssembler()
  .setInputCols(Array("marks"))
  .setOutputCol("features")

val d = a.transform(data)

val model = new LogisticRegression()
  .setLabelCol("label")
  .setFeaturesCol("features")

val result = model.fit(d)

val prediction = result.transform(d)

prediction.select("marks", "label", "prediction").show()

spark.stop()
}
```

Output:

marks	label	prediction
1.0	0.0	0.0
2.0	0.0	0.0
3.0	0.0	0.0
7.0	1.0	1.0
8.0	1.0	1.0
9.0	1.0	1.0